

Da captura de imagens à exibição de modelos 3D

*Guia prático para modelagem do acervo do
Museu Histórico de São José*

FOTOGRAMETRIA E MODELAGEM

1. Captura de Imagens – A Base da Fotogrametria

A captura de imagens é a etapa fundamental para o sucesso da reconstrução 3D. Deve-se utilizar uma **câmera estável** (DSLR ou celular com boa lente, no nosso teste utilizamos ambos os tipos e um celular do modelo iphone 16e teve uma performance melhor que a câmera no momento da fotogrametria). O objeto deve ser fotografado em **360°**, com sobreposição entre as imagens, cobrindo todos os ângulos possíveis: frente, laterais, traseira, topo e, se possível, a parte inferior. As fotos devem ser tiradas com a câmera em movimento suave e constante, evitando borrões. Utilize **luz difusa**, como em dias nublados ou na sombra, evitando luz solar direta, que gera sombras e compromete as texturas.

O número de fotos recomendado varia: no mínimo **30**, idealmente entre **50 e 100**, e preferencialmente entre **150 e 250** em alta resolução. Após a captura, organize todas as imagens em uma **pasta dedicada** para facilitar o processo posterior.

2. Reconstrução do Objeto 3D com Meshroom

O **Meshroom** é o software utilizado para processar as imagens via fotogrametria. Após importar as fotos (arrastar para a área central), o usuário deve escolher um diretório de saída e clicar em **'Start'**. O Meshroom executa um pipeline automático para reconstruir a malha 3D com base nos pontos em comum entre as fotos. Esse processo pode ser demorado dependendo da quantidade de fotos e da capacidade do computador.

Ao final do processo, o modelo pode ser visualizado no nó **'Texturing'**. O arquivo gerado estará na pasta MeshroomCache > Texturing, com extensão **.obj**. Salve o objeto 3D para utilizá-lo nas próximas etapas.

3. Limpeza e Escultura no Blender

Abra o **Blender**, crie um novo projeto e delete os objetos padrões da cena. Importe o modelo **.obj** gerado pelo Meshroom. Ative o modo **'Material Preview'** para visualizar as texturas. Em seguida, entre no modo **'Wireframe'** e vá para **'Edit Mode'**. Selecione o objeto principal, inverta a seleção e delete os resíduos (vértices indesejados), repetindo até que reste apenas o objeto desejado.

Utilize o modo **'Sculpt Mode'** com o pincel **'Smooth'** para suavizar a malha, eliminando imperfeições visuais. Após a escultura, exporte o modelo limpo novamente como **.obj**.

4. Regularização e Otimização da Malha com Instant Meshes

O arquivo **.obj** exportado do Meshroom ou do Blender deve ser importado no **Instant Meshes**. O objetivo é regularizar a malha de polígonos, que costuma ser irregular e densa. Use as ferramentas para remapear os polígonos, tornando-os mais uniformes (quadriculados ou triangulares). A configuração recomendada para remapeamento é o modo **'triangles'**, com contagem entre **8.000 e 10.000 vértices**, o que garante um equilíbrio entre leveza e qualidade visual.

Após aplicar os ajustes, clique em “Solve” duas vezes e exporte o novo modelo otimizado no formato **.obj**.

5. Ajustes Finais e Bake de Texturas no Blender

Importe no **Blender** tanto a malha otimizada quanto a original, posicionando ambas nas mesmas coordenadas (0,0,0). Aplique **UV Mapping** na malha leve usando o recurso **'Smart UV Project'**, ajustando a margem para 0.01. No Shader Editor, crie um material e adicione dois nós de imagem: um para a textura diffuse (cor) e outro para o mapa de normais (relevo). Crie imagens **2K**: uma com alpha ativado para o diffuse, outra com 32 bits e alpha desativado para o normal.

Na aba de renderização, ative o motor **Cycles**, acesse o menu **'Bake'**, selecione o tipo **'Diffuse'**, marque **'Selected to Active'** e configure a extrusão para 0.01. Clique em **'Bake'**. Salve a imagem gerada na aba **'UV Editing'**. Repita o processo para o mapa de normais.

6. Modelo Final para Aplicações

O resultado final consiste em dois modelos: um altamente detalhado e pesado, e outro leve e otimizado para uso em tempo real. Graças ao processo de baking de texturas, o modelo leve mantém uma aparência visual rica. Ele é ideal para aplicações interativas em 3D, como jogos, realidade virtual e experiências na web.

Esse processo completo utiliza apenas softwares gratuitos: **Meshroom** para digitalização, **Blender** para escultura, otimização e texturização, e **Instant Meshes** para regularização.

EXIBIÇÃO / PUBLICAÇÃO

Esta seção do manual detalha o processo para tornar os objetos 3D acessíveis e visíveis online. Apresentaremos os dois métodos principais utilizados: a publicação no Firebase

para obter um link de acesso direto e a execução do projeto em um container Docker na máquina virtual (VM) do IFSC.

Links de Referência:

- Repositório do Projeto (GitHub): <https://github.com/byte-with-a-dot/museu-3D>
- Teste de exibição com GitHub Pages: <https://00thiago.github.io/onlineMaca3D/>
- Exibições com Firebase hosting:
 - <https://maca3d-2d1eb.web.app/> (maçã teste)
 - <https://zoolito-8d246.web.app/> (zoolito)
 - <https://machado-c2267.web.app/> (machado)

1. Preparando os Arquivos de Visualização

Todos os arquivos necessários estão disponíveis em nosso repositório no GitHub.

- **index.html:** Este é o arquivo HTML principal que cria a estrutura da página onde o objeto 3D será exibido. Ele contém um elemento canvas (`document.body.appendChild(renderer.domElement);`) que o Three.js utiliza para renderizar a cena.
- **main.js:** Este arquivo JavaScript contém toda a lógica para carregar o modelo 3D (usando os módulos *OBJLoader* e *MTLLoader* ou o módulo *GLTFLoader*), configurar a cena, a câmera, a iluminação e os controles de interação do usuário (como girar e ampliar o modelo).

- **Modelo 3D:**

Método 1: Formato *.obj* com *.mtl* (Utilizado neste Projeto)

*Este é o método que implementamos para exibir os modelos atuais. Ele utiliza dois arquivos que trabalham em conjunto e requer os módulos *MTLLoader* e *OBJLoader* do [Three.js](https://threejs.org/).*

- *.obj*: Contém a **geometria** do modelo (os vértices e faces que compõem sua forma 3D).
- *.mtl* (Material Library): Contém as informações sobre os materiais e texturas. É ele quem diz ao renderizador qual cor ou imagem aplicar em cada parte da geometria definida no arquivo *.obj*.

Como usar: Para carregar um modelo neste formato, você precisará editar os caminhos dos arquivos nas duas funções separadas dentro do *main.js*:
mtlLoader.load() e *objLoader.load()*.

Método 2: Formato *.glb* (Alternativa Recomendada)

Este é um formato mais moderno e otimizado para a web. Sua vantagem é ser um arquivo binário único que já inclui tudo: a geometria, os materiais e as texturas. Para carregá-lo, utiliza-se o módulo *GLTFLoader* do [Three.js](https://threejs.org/).

- *.glb*: Contém o modelo 3D completo em um só arquivo. Isso simplifica o gerenciamento dos arquivos e geralmente resulta em um carregamento mais rápido na página.

Como usar: Para carregar um modelo `.glb`, você precisaria comentar ou remover o código referente aos loaders de `.obj` e `.mtl` e usar o `GLTFLoader`, ajustando o caminho do arquivo na sua respectiva função de `load()`.

- **Bibliotecas Three.js:** Utilizamos a biblioteca Three.js para renderizar o objeto 3D no navegador. É possível utilizar a mesma estrutura do arquivo `index.html`. Também é possível encontrar boa documentação e exemplos no [manual oficial do Three.js](#) e na [página de exemplos](#).

2. Publicação com Firebase Hosting

O Firebase Hosting oferece uma maneira rápida, segura e simples de hospedar um projeto web, gerando um link compartilhável. Este método é ideal para testes rápidos, validação do cliente e para gerar um `iframe` para incorporação em outros sites ou para acesso público sem a necessidade de gerenciar um servidor.

2.1. Configuração Inicial do Firebase

1. **Criar Conta e Projeto no Firebase:** Acesse o console do Firebase, faça login com uma conta Google e crie um novo projeto.
2. **Instalar o Firebase CLI:** É necessário ter o Node.js instalado em sua máquina. No terminal, execute: `npm install -g firebase-tools`
3. **Autenticar e Inicializar:**
 - a. Faça login no Firebase pelo terminal: `firebase login`.
 - b. Navegue até o diretório raiz do seu projeto (onde está o `index.html`) e inicialize o Firebase: `firebase init`.
 - c. Durante a inicialização, selecione **Hosting**. Escolha o projeto Firebase que você criou e configure seu diretório público (normalmente `public` ou apenas a raiz `./`). Responda **"Não"** à pergunta sobre configurar como um `single-page app`.

2.2. Implantando o Projeto

Após a inicialização, um arquivo `firebase.json` será criado. Para publicar seu site, execute o comando a partir da raiz do projeto: `firebase deploy`

Ao concluir, o terminal exibirá a URL onde seu projeto está hospedado.

3. Publicação na VM do IFSC com Docker

Para integração com os sites existentes do museu e para garantir que o projeto seja executado em um ambiente controlado, utilizamos um container Docker na máquina virtual do IFSC. O Docker nos permite empacotar a aplicação com todas as suas dependências, garantindo que ela funcione de forma consistente em qualquer lugar.

3.1. Acessando a VM

Primeiramente, conecte-se à VM do IFSC. As credenciais e os passos iniciais estão detalhados na documentação interna (`docTesteVM.md`).

- Servidor/Hostname: **TELE-Eraldo-ADS-Extensao**

- Usuário: **equipetresd**
- Comando de Conexão:
`ssh -i ./id_rsa equipetresd@extensaoads.sj.ifsc.edu.br`

Dica importante: Garanta que sua chave SSH (id_rsa) esteja no diretório correto e com as permissões adequadas, conforme o guia de configuração inicial disponível no Trello ("Sprint 3") ou no documento **docTesteVM.md**.

3.2. Clonando o Projeto para a VM

Uma vez conectado à VM, clone os arquivos do projeto do repositório para o diretório apropriado. Uma alternativa é usar o comando `scp` para copiar arquivos locais para a VM.

Exemplo de cópia de um diretório local para a VM:

```
scp -r /caminho/local/arquivo
equipetresd@TELE-Eraldo-ADS-Extensao:/srv/comum/projeto3D
```

3.3. Utilizando o Docker

Nosso projeto inclui um **Dockerfile** que automatiza a criação de uma imagem de container com um servidor Nginx para servir os arquivos estáticos.

1. **Construir a Imagem Docker:** Navegue até o diretório raiz do projeto e execute o comando de build: `docker build -t online-maca-3d .` (o nome do objeto deverá ser alterado).
2. **Executar o Container Docker:** Após a construção da imagem, execute o contêiner, mapeando a porta 80 do contêiner para uma porta na VM (ex: 8080): `docker run -d -p 8080:80 online-maca-3d`
 - a. **-d** executa o contêiner em modo "detached" (em segundo plano).
 - b. **-p 8080:80** mapeia a porta 8080 do host (VM) para a porta 80 do contêiner.
3. **Acessando o Objeto:** O objeto 3D estará então acessível através do endereço de IP ou hostname da VM na porta mapeada (ex: `http://extensaoads.sj.ifsc.edu.br:8080`).

Mais detalhes sobre os comandos do Docker e gerenciamento do contêiner podem ser obtidos no arquivo **README-docker.md** no repositório.

4. Recomendações Finais

- **Controle de Versão:** Sempre utilize Git. Antes de fazer alterações significativas, crie uma nova *branch*. Isso facilita o rastreamento das modificações e a reversão para um estado anterior, se necessário. (não utilizamos branches em nossa atividade e tivemos algumas dificuldades)
- **Documentação:** Ao fazer alterações ou adicionar novos recursos, atualize a documentação. O arquivo **log_uso.txt** no diretório comum (*/home/publico*) da VM foi pensado como bom local para deixar notas rápidas para o restante da equipe.
- **Otimização:** Carregamentos mais rápidos e interações mais suaves com modelos 3D na web são condicionados, também, por objetos com baixa contagem de polígonos e texturas comprimidas

